

Measured Models: A Hardware Perspective on Non-Bypassable AI Alignment

Thomas Zhang Haiyue Ma David Wentzlaff
Department of Electrical and Computer Engineering
Princeton University, Princeton, NJ, USA

Abstract—Runtime safety mechanisms based on model activations such as linear probes, sparse autoencoders, and activation steering can expose safety-relevant signals that are invisible to input/output filters. Their enforcement, however, is normally delegated to the same software stack that launches the model. That assumption is fragile for autonomous agents and malicious operators: the application can omit the checker, alter its inputs, ignore its verdict, or execute a different model entirely. This paper proposes a GPU hardware root of trust for non-bypassable alignment *monitoring*. We develop a threat model, compare software, hypervisor, and GPU-rooted trust models, and establish the necessity of a restrictive signed-manifest execution protocol that binds model weights, checker parameters, and kernel code. We then outline a Vortex-based prototype comprising an ingress hashing engine, immutable verified memory regions, an execution-state machine, and a sparse-autoencoder checker accelerator.

I. INTRODUCTION

The increasing capabilities of AI have given rise to concerns about misalignment, which is when the AI produces outputs that are not in accordance with human values. Various measures have been taken to counteract misalignment, including training-time methods like reinforcement learning from human feedback (RLHF) and inference-time methods such as prompt optimization and input/output gating. Such end-to-end methods, however, are often too coarse: they may over-bias models toward refusal or fail to detect misalignment cases that fall outside the training distribution. Moreover, an AI might produce seemingly harmless outputs while its internal reasoning or intentions are dangerous.

As such, researchers have begun trying to inspect the model’s internal activations to more surgically detect misalignment before a new token is even generated. Linear probes have detected deceptive behavior from hidden states [1]; sparse autoencoders (SAEs) can decompose activations into more interpretable features [2]; and activation-steering methods alter residual-stream representations during the forward pass [3]. SafeSwitch combines an internal-state monitor with a conditional safety intervention [4]. These methods show great promise, demonstrating that model internals contain actionable signals about safety-relevant behavior that can catch misaligned behaviors that would not have been caught with tuning and I/O filtering.

However, the advent of agentic AI has increasingly enabled models to initiate actions autonomously and modify aspects of their own runtime execution environments, raising the question of whether we can reliably trust such models to run safety checks on themselves. Technologies like OpenClaw grant AI systems the ability to interact directly with files, applications, and operating-system interfaces on the host machine. Emerging agent frameworks increasingly allow AI systems to iterate on their own prompts, tools, code, and execution environments. Furthermore, as open-weight models become more and more capable, it is almost certain that bad human actors will misuse them. A person or agent may simply choose not to run a software ‘safety checker’ alongside their model. It is desirable, therefore, to introduce a root of trust that guarantees the enforcement of alignment on AI models. Our proposal is to place the root of trust in hardware. That is, if an AI model is to run on a piece of hardware with our root-of-trust, then it has no choice but to be subject to a misalignment monitor. In this text, we wish to explore whether or not adding hardware support for AI misalignment checking is feasible, or worthwhile, to do, and the potential ways in which such a mechanism could work.

This paper makes the following contributions:

- We define a threat model for non-bypassable activation monitoring and compare three placements of the trusted computing base (TCB).
- We propose a signed-manifest protocol that binds model weights, checker parameters, executable kernels, memory ranges, and launch metadata.
- We present a Vortex GPU implementation with ingress hashing, protected memory regions, and an sparse autoencoder (SAE) checker accelerator.
- We analyze flexibility and security tradeoffs, emphasizing the difficulty of verifying model behavior through hardware primitives.

II. TRUST MODEL

As previously mentioned, existing alignment techniques, such as activation steering and probe-based methods, rely on the assumption that the application launching the model will correctly invoke the checking mechanism and respect its verdict. This assumption becomes void if the application is itself a highly autonomous or agentic workload. We wish

to be rid of this assumption. Figure 1 illustrates three possible trust models.

Software-Rooted enforcement. In the first model, shown on the left, the trusted computing base (TCB) includes the cloud CPU hardware, the hypervisor, and the guest applications running the AI workload. To make this model defensible, we would need to closely limit the abilities of the cloud applications. The primary function of an application should be to take a prompt, execute an approved model with approved kernels, run the checker, and release the output only if the checker allows. Thus, agentic/autonomous behavior would be limited to the “API callers” outside of the cloud. For example, OpenClaw runs on a PC, and it sends requests to the rigid cloud API whenever it needs to perform model inference. As such, we can think of the cloud application as a sort of ‘static graph’, where the checker is built into the graph.

This is a difficult trust model to defend, because the TCB here is vast and easily mutable. The TCB includes not only the hardware/hypervisor, but also the guest OS, model runtime, device drivers, and the application responsible for running the checker. Any vulnerability or malicious modification in this large and dynamic software stack might disable the checker, bypass it, or fabricate a successful result even if the checker flagged the session. The cloud provider would have to verify that every AI workload uses a foolproof guest image and inference path. There is also the risk of a malicious operator deploying a different guest application altogether. Thus the ‘policy enforcer’ (e.g. a governmental entity) would have to continuously audit the applications running on the cloud and verify that they are foolproof; this is unrealistic.

Hypervisor-Rooted enforcement. For context, AI workloads today largely run as guest applications on Virtual Machines in the cloud, where bare-metal hypervisors are used as means of isolation and portability. The second trust model, proposed by Guillotine, moves enforcement into a trusted hypervisor [5]. In this configuration, the model runs in an untrusted guest environment, but has to talk through a trusted hypervisor to interact with I/O devices and the outside world. Whenever the guest application wishes to interact with the network card, or GPUs, it has to send a request through the ‘port API’ of the hypervisor, which reviews the activity in its own software and decides whether or not it is malicious. However, the trusted computing base remains large, encompassing the bulky hypervisor software, processor logic, device drivers, and the large ‘port API’. The platform would need to be rigorously verified to reduce the risk of VM escape or side-channels.

GPU-Rooted enforcement. The last trust model that we consider is to only trust the GPU hardware (and whatever firmware/software is securely verified by the hardware). If we are able to guarantee the enforcement of AI alignment checking only on the GPU end, then we could greatly reduce the size of the trusted computing base, and greatly reduce the risk of exploits and side channels.

Moreover, it is more difficult for bad actors to ‘switch-out’ or tamper with valuable GPU hardware. We would achieve a smaller and more impenetrable attack surface. Furthermore, if we place the checker on the GPU-side, we have the highest level of visibility and highest available throughput to inspect the GPU-resident model state that we need for the purposes of our checker. The GPU would perform detection and mitigation on-die, only releasing an authenticated result only after an approved checker had examined the relevant execution and permitted the output.

III. MEASURING THE MODEL

If we are to go with the rightmost trust model, we need to enforce the non-bypassability of a checker within the GPGPU hardware. That is, we would like to make it so that any model that runs on the hardware (which we take to be a GPGPU) must be subject to a checking mechanism. If we are to do this, there are a number of practical challenges to overcome. For example, how does the hardware know which model is running on the GPGPU? How does it know where and when to tap the hidden layer to inspect the activations? Where can the checker store its weights/data so that they are safe and isolated from outside inspection? Ideally, we would not like to trust anything in the software domain, including the GPU driver, the host CPU process that launches the kernel, and the kernel running on the GPGPU itself. These are all things that an extremely capable AI agent, or a malevolent user, may be able to tamper with.

It would be extremely difficult for a chip, using just what it sees from hardware (memory accesses, DMA streams, register values, tensor-core instructions), to reverse-engineer the model and understand its structure and perform the appropriate checker mechanism. Certainly, it is impossible for the chip to generate the appropriate checker weights (which are normally learned through supervised/unsupervised learning) on the spot to semantically inspect a model it has never seen before. One solution is to have a whitelist of approved models: a list of models that the chip is allowed to run. We could store the checker weights for all of these models in an on-chip secure repository. If the hardware recognizes the model and it is on the whitelist, then it can simply draw from the checker weight repository. If it does not recognize the model, it is not allowed to run.

There is still the issue of how to recognize the model. Existing LLM fingerprinting techniques (developed for Ownership and IP protection purposes) use precise watermarking techniques which condition the model to respond a certain way when prompted with certain trigger inputs [6]. These methods are difficult to use in our case because the hardware, without support from the software side, cannot directly call the model API. An alternative method would be to hash the model weights as they come in and compare the hash with a hardware lookup table of model hashes (or send it through a NIC to something like a trusted remote attestation service). Another option would be to

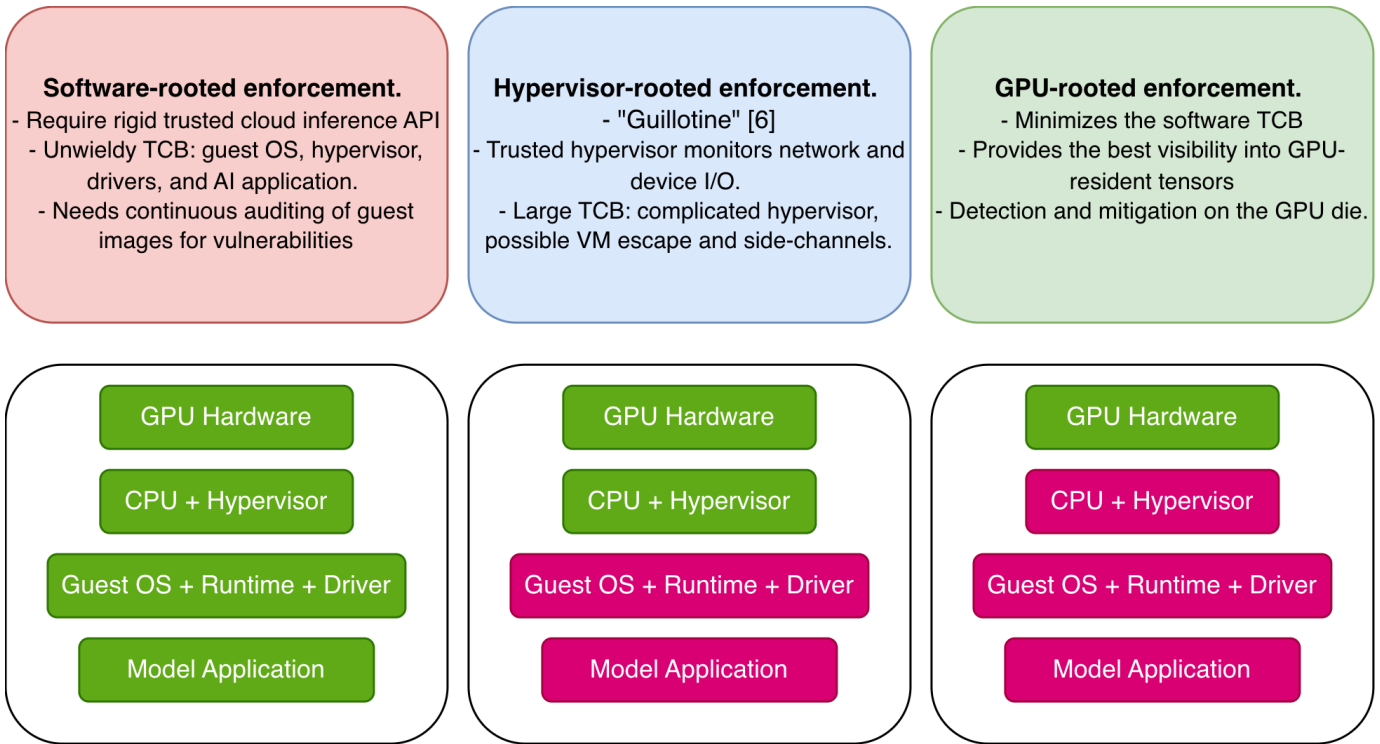


Fig. 1. Three possible trust models for AI alignment enforcement.

use an asymmetric cryptographic key pair and allow the model weights to be transferred to VRAM with a digital signature, and then hash the weights to see if they match the decrypted signature. This presumes that a trusted third party entity has created a database of ‘trusted’ models and produced digital signatures for each using a secret private key. As such, only models that are paired with a valid signature can run on the hardware. In fact, we could send the checker weights, model weights, and a report of the address ranges of the locations of each layer together through DMA to the hardware, and sign the combined image with a digital signature. With this, the hardware doesn’t have to derive the checker weights or model structure on its own.

It is important to note that embedding high-throughput cryptographic hardware onto the GPU die isn’t unprecedented. NVIDIA’s Confidential Computing Mode, for example, enforces that weights are encrypted as they are transferred over PCIe and then decrypted on-chip through dedicated cryptographic circuits (AES-256), with minimal overhead. Our hashing mechanism could be done in parallel with the workload itself using a dedicated hash engine, killing the workload upon verification failure. As another side note, hardware LLM fingerprinting mechanisms like ingress hashing might also be useful for compute governance: chip manufacturers or governments might want to limit the models that can be run on domestically produced chips. Even if chips are smuggled or stolen, they could be made to only be able to run domestic, closed-source

models. Moreover, the government may want to control the development/deployment of AI by maintaining a list of whitelisted models that are allowed to run on GPU hardware.

There are still a number of glaring issues with the above solution. How do we know that the kernel code will actually use the weights that were loaded and verified, and that it will use those weights exactly as expected? To know this, we need to know what the kernel code is actually doing. We may be able to use some heuristic such as measuring the proportion of loads or cache misses that come from the model weight address range. This, however, misses the fact that a large portion of loads and cache misses can be attributed to the KV Cache, whose memory traffic can become just as large as the model weights as context grows. We could also limit the amount of non-model data that is transferred to VRAM to prevent the user from trying to run some other model. This could prove restrictive when we are trying to move large amounts of dynamic data between GPUs (e.g. the KV cache if we are doing prefill-decode disaggregation). Another solution might be to bring the entire cluster of GPUs under the trusted computing base. This way, we can allow the GPUs to transfer large amounts of data between each other freely (e.g. through NVLink), and provide only a small budget for the amount of unhashed data that can be transferred from CPU to GPU (for kernel code and input token index arrays). However, input tokens are a continuous stream with no ‘limit’ during a given runtime; we can’t easily put a hard cap on them. The host

TABLE I
PITFALLS IN VERIFYING KERNEL BEHAVIOR WITHOUT BINDING THE FULL EXECUTION.

Requirement	Candidate mechanism	Why it is insufficient
Prove verified weights are used	Require a fraction of loads or misses to target verified ranges	KV-cache and activation traffic confound any fixed ratio; a kernel can generate fake accesses.
Prevent a second model	Limit CPU–GPU imports to weights, code, and token IDs	A host can gradually smuggle parameters through an unbounded input stream or use free HBM capacity for a smaller model.
Locate and order the hidden state	Track MAC operands and result provenance	Requires tags through registers, shared/local memory, moves, copies, arithmetic identities (e.g. add 0), and tensor-core operand staging; the kernel can make a copy of the weights so that the hardware can’t see when they are accessed; the kernel can multiply a dummy hidden layer by the weight matrix that we are monitoring to trick the checker into checking the wrong layer.
	Allow approved kernels exclusive access to verified QKV projection weights so that the application is forced to provide the correct hidden layer	The application can avoid those weights by running a different model, or recover/copy them through algebraic queries unless the entire execution is constrained.
Detect when the hidden state is ready	Trigger on the next layer’s weight reads	An untrusted kernel can issue early fake reads or MACs and present an incomplete or decoy tensor.

could just gradually ‘smuggle’ a different model in through the input token channel.

There is also the issue of locating the activation layer that we tap for our checker. We could ask the user to report the activation layer address range in their signed model image along with everything else. But an adversarial kernel might save the activations to a different place altogether, or it could jumble and mix up the order of the activation values within the layer so that our checker fails to function. We could try to do some data-tracing — tracking what each activation is multiplied by in the verified weight image — but this would likely require complicated hardware support. For example, if we are using a GPU with tensor cores, we could require that whenever one operand is sourced from the activation layer, the other operand is sourced from the corresponding weight matrix for that layer. We will likely need to be even more precise than that, though. We must track the operands at the scalar level to verify that each activation in the activation layer is multiplied by the correct weight (to make sure that the order of the activations isn’t jumbled in memory). Another issue is that GPU tensor cores typically read from general purpose register files because of the bandwidth requirements of MMA units, requiring us to keep track of load addresses as per-register metadata. Moreover, kernels often cache data within local and shared memory to improve locality; we would need to add tracking metadata to those data structures too. We will also likely need to track source addresses across instructions like ‘add 0’ and ‘mov’ and to prevent the “laundering” of the data provenance. A malicious kernel could, for example, make a copy of the weights to a different address region

so that our hardware doesn’t know what model weights it is accessing. A malicious kernel could also multiply the weight matrix by dummy values to mislead the checker into thinking that a ‘fake’ hidden layer is the real one.

IV. HASHING THE KERNEL CODE

A key lesson is that whatever restraint that the hardware imposes on the kernel execution, it is likely that a malicious kernel can ‘fake’ adherence. To ameliorate some of this complexity, we should hash the kernel code and kernel arguments alongside the rest of the model. Any kernel that runs on the GPGPU will be hashed alongside the model weights, checker weights, model structure, and then verified against a cryptographic signature. A large disadvantage to the approach of hashing everything is inflexibility and non-configurability. Every time one wants to change the kernel code, change the precision of the weights, change the address of the weights, change the address of the input and output matrices, or change the kernel arguments (e.g. tile size, etc), one has to submit their new configurations to a remote trusted third party to confirm and sign. Because we are hashing raw machine code, even if the kernel code is compiled slightly differently, it will need to be re-signed. The cryptographic signing of software and firmware is already pervasive in modern chips, such as during ‘secure boot,’ where bootloaders and OS kernels are verified against cryptographic signatures before they are loaded into RAM and executed. In NVIDIA’s GPU-CC, GPU firmware is often hashed, signed, and sent to a remote server (NRAS) for verification during boot in a process called "remote attestation" [7].

At this point, since we are hashing the kernel code and kernel arguments, we have two options: we can implement the semantic checker as a dedicated accelerator in hardware, or we can embed the checker mechanism in the verified kernel software. If we do the latter, our hardware reduces to a hashing and decryption mechanism. Using purely software may require us to use a fused kernel; we should fuse the checker algorithm with another essential kernel (e.g. the unembedding GEMM kernel, or a QKV projection kernel). Otherwise, if we model the checker software as a separate kernel, then the untrusted host might simply choose not to launch the checker kernel.

To be sure that the GPU actually only executes the verified kernel, we could mark all non-kernel pages as non-executable within the GPU’s MMU. Another thing to address is the possibility that the host CPU might continue issuing DMAs to overwrite the verified weights or the verified kernel code. How do we protect against ingress routes like NVLink and DMA from altering our verified kernels and weights? We can lock verified weight regions until the next load of verified weights, and we can lock verified kernel code regions until the next verified load of kernel code. Locking, in this case, would be making the regions of physical memory read-only to all ingress routes. The gating of writes would likely be placed deep within the memory fabric itself, to be certain that all ingress routes are blocked.

I’ve summarized my reasoning for these methods of enforcement (hashing the weights, the kernel code, and the kernel arguments) in Figure 2. The central challenge is proving that the checker observed the computation that produced the output. Table I summarizes tempting alternatives that avoid whole-kernel measurement.

V. DESIGN REQUIREMENTS

A GPU-rooted design should satisfy the following requirements.

R1: Artifact identity. The GPU must authenticate the model, checker, executable kernels, and security-critical metadata.

R2: Immutable verified state. After verification, model weights, checker parameters, and executable code must be read-only to all ingress paths, including PCIe DMA, peer-to-peer DMA, and GPU interconnect writes.

R3: Executable-code confinement. The GPU must ensure the execution of the verified kernel code pages and prevent execution from unsigned pages.

R4: Correct activation binding. The checker must consume the intended tensor in the intended element order and at the intended program point. An adversarial kernel must not be able to present a decoy tensor.

R5: Deployability. The mechanism should support batching, tensor parallelism, kernel fusion, and multi-GPU execution without requiring a new hardware revision for every new software configuration.

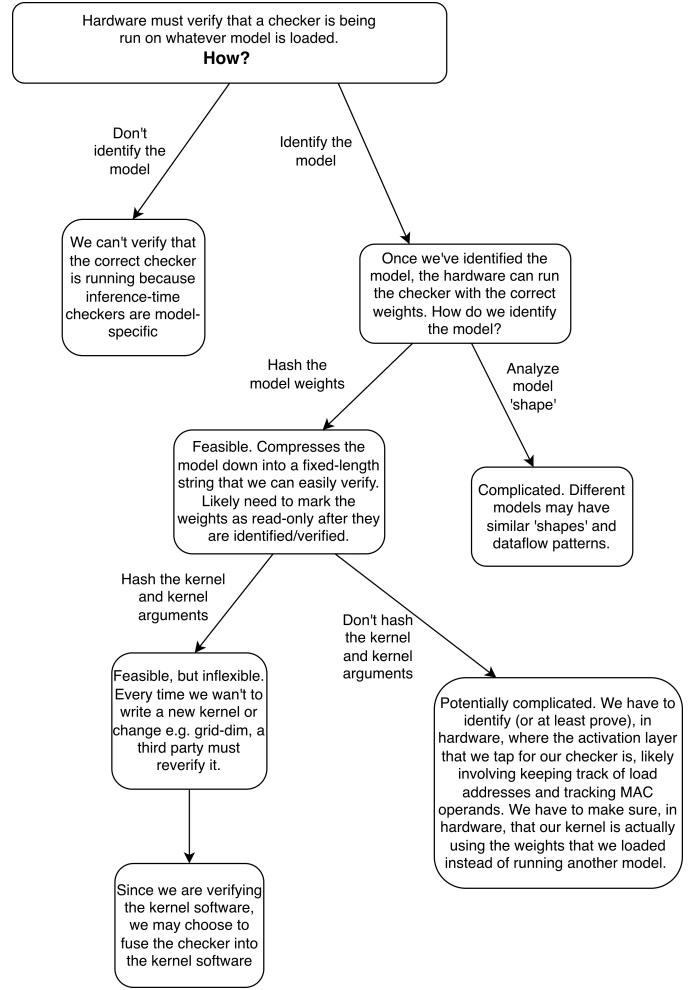


Fig. 2. Flow-Chart of Possible Methods of Enforcement.

VI. MEASURED MODEL ARCHITECTURE

Figure 3 shows a possible end-to-end design for the admission mechanism and Launch FSM, integrated with a hardware SAE accelerator and ingress hashing.

A. Signed Manifest

The host first transmits a signed manifest describing the complete admitted configuration. A manifest could include:

- cryptographic hashes of model shards, checker parameters, and permitted kernel code images;
- virtual address ranges for weights, kernel code, activation tensors, and outputs;
- kernel entry points and arguments (e.g. pointers to kernel inputs/output buffers)
- the checker trigger policy

The manifest is signed by a trusted authority whose public key is provisioned in ROM or one-time-programmable storage.

B. Admission and Launch FSM

When the GPU boots, it is in the *Load Manifest* state. During this state, instruction fetch is disallowed. The host

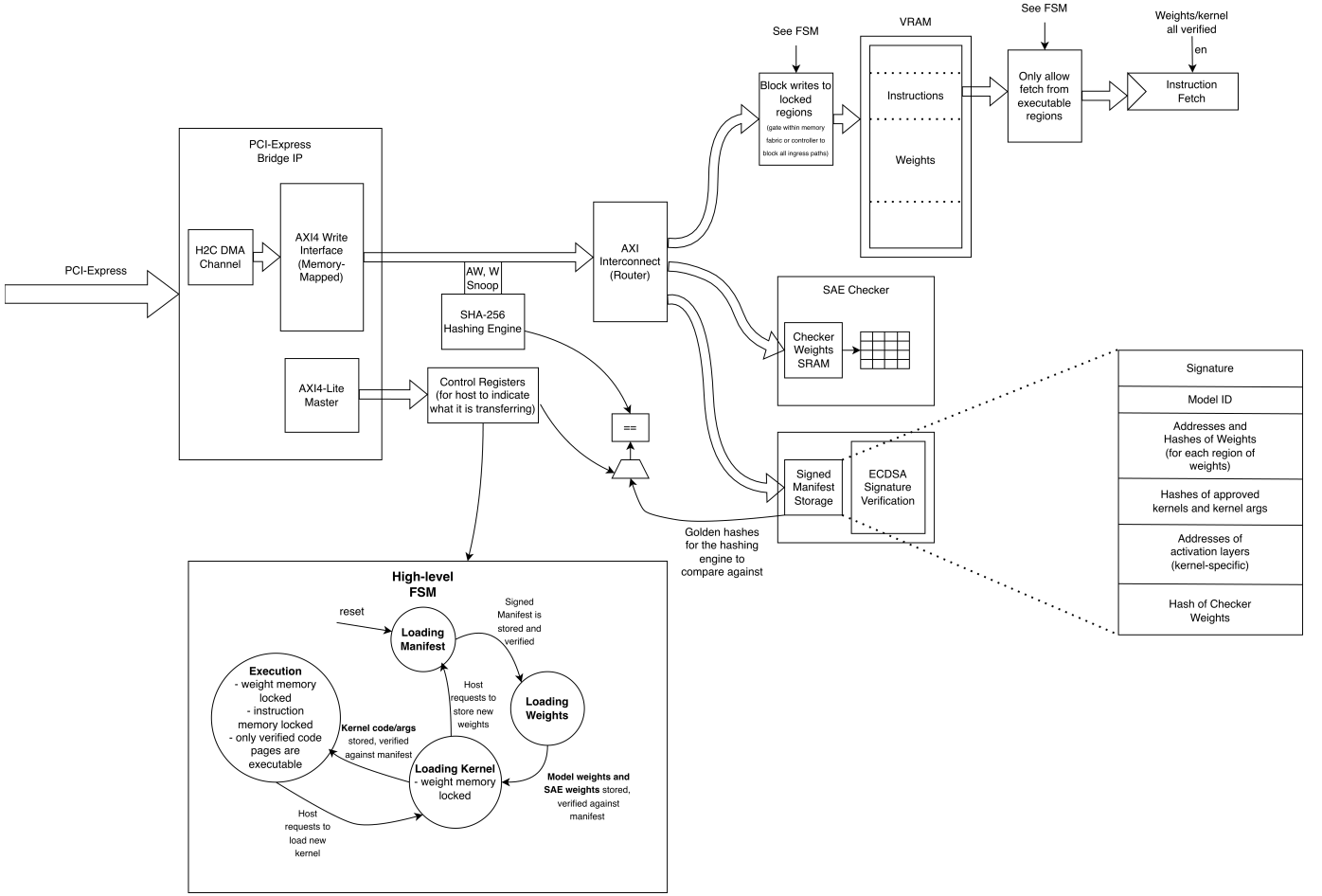


Fig. 3. End-to-End Protocol for Non-Bypassable Checker Mechanism.

must transmit a signed manifest, which is routed to a small manifest storage unit, where it is verified against its signature using ECDSA Signature Verification (using a public key that is burned onto ROM or an OTP fuse). After verification of the manifest, we enter the *Load Weights* state, in which the model weights and checker weights are transmitted through the on-chip interconnect to VRAM and the protected Checker RAM, respectively. The host can set control registers to indicate what it is about to transmit. As the weights are transmitted to VRAM and the Checker SRAM, they are hashed on the fly and compared to the golden hashes stored in the manifest. After the weights are verified, we enter the *Load Kernels* state, in which the weight memory is locked (made unwritable to all possible ingress routes). In this state, the kernel is loaded into VRAM via the on-chip interconnect (e.g. AXI), where it is hashed and verified against the manifest on the fly. Once it has been loaded, we enter the *Execute* state, where the kernel code pages are made read-only, all other pages are made non-executable, and execution is allowed to start at the manifest-specified kernel entry points.

C. Memory Protection and Ingress Control

Protection must be enforced below individual DMA engines. Certain physical memory regions might be tagged with policy bits such as **READONLY**, and **EXECUTABLE**. Every write requester—PCIe, NVLink, copy engines, and SM stores—is checked at the memory fabric. This avoids a design in which one ingress route accidentally bypasses a page-table check.

The checker repository can be implemented as protected SRAM or an protected HBM region. Storage is not negligible: F selected FP16 SAE features at hidden width d require $2dF$ bytes. For $d = 4096$ and $F = 1024$, the encoder alone is 8 MiB. A practical implementation may use fewer features, INT8/FP8 weights, or protected HBM region with an on-chip cache.

D. Kernel Binding and Flexibility

Hashing raw machine code maximizes assurance but reduces configurability. Recompile, tile-size changes, address relocation of weights/buffers, or differences in quantization produce a new measurement. Certain strategies might reduce signing churn:

- 1) sign position-independent kernels and dynamically ensure that specified kernel arguments (e.g. pointers to input/output buffers) match the addresses of verified weights/buffers;
- 2) delegate compilation of newly-written kernels to the trusted third party, which will run a more sophisticated (i.e. extensive operand tracing), possibly automated, procedure to verify and sign the kernel.

The first prototype uses whole-kernel measurement because it is easier to reason about; later systems should explore more flexible options.

This solution (to hash virtually everything that enters the GPU such that we have approved code running on approved weights) might seem similar in spirit to the sort of ‘run model X on prompt Y’ interface that we illustrated in the first trust model (see Section 2). The difference is that enforcement is applied at the GPU boundary, a much smaller attack surface than the cloud software stack.

E. Host Orchestration Attacks

One might also ask if it is enough to verify individual software kernels, or if it is also necessary to verify the code that is on the host CPU that launches and manages the kernels. After all, large language models have become extremely complicated beasts, sharded across many GPUs and orchestrated with sophisticated interconnect protocols and host scheduling. That is, even if we make it such that the GPU is only capable of launching approved kernels on approved weights, can the host CPU still attack the functionality of our checker?

For example, the host CPU might choose to skip the model layer that contains the kernel with the checker embedded into it. Or, it could perform that layer on the CPU, accepting the extra computational cost. We could combat this by embedding checkers into as many layers as possible, making it impossible for a malicious host to skip all of those layers without rendering the forward pass completely useless or impractically slow. The host could also perform some procedure to try to fine-tune the input activations in order to pass the checker. But this issue comes down to the efficacy of the checker itself (i.e. whether the set of passing inputs includes any that produce an unsafe output). This text only aims to enforce non-bypassability, not checker infallibility.

VII. HARDWARE CHECKER PROTOTYPE

A. SAE Accelerator Datapath

In Figure 4, we have illustrated a design in which the checker arithmetic is performed in hardware in a small accelerator. Here, we explore what such an accelerator could actually look like. For this purpose, we have chosen to implement a sparse autoencoder (SAE) probe. A sparse autoencoder is a small neural network trained, using unsupervised learning, to decompose a model’s hidden layer activations into a larger set of sparse, interpretable features (Cunningham et. al. 2023). The trained SAE model consists

of an encoding matrix and a decoding matrix, trained together on a specific hidden layer within the model. For a given token, we can multiply the activations produced from that hidden layer by the encoding matrix to retrieve a score for each SAE feature. However, we do not have to use the entire encoding matrix; we can select only the set of features that are most indicative of misalignment to use in our probe. To detect misalignment, we compare these feature scores to a set of tuned thresholds; if a certain number of these features exceed their thresholds, then we flag the token.

Our goal is to build a hardware implementation of the SAE probe that computes, in parallel with the forward pass of the LLM, the scores for the selected SAE features. We build this prototype on top of the open-source Vortex GPGPU, configured to have a memory hierarchy similar to that of NVIDIA GPUs. Each Vortex core (analogous to an SM) has its own L1 cache, while a unified L2 cache is shared between all of the cores. The checker snoops the read ports of the L2 cache and triggers the checking logic when any core reads an address within a certain weight region of the model. We also provide the option of triggering the checker as soon as kernel execution begins. This method presumes that the desired activation layer values are present from a previous kernel that was resident on the GPU. As soon as the checker is triggered, the activations are read into the checker through a dedicated virtual port on the L2 cache. The reasoning for this is that since the activations were written to L2 in a previous kernel (we model the L1 cache as write-through), the desired values are very likely to be cached in L2, minimizing the latency of the checker reads.

It is important to note that the port that we added onto L2 is not a physical read port (which would greatly increase the area footprint of the cache), but simply another requester on the cache’s arbiter. The arbiter distributes L2 requests among the single-ported banks, time-multiplexing as needed. To amortize the latency of L2 requests, each row has a buffer with the capacity of two cache lines. The buffer is “double-buffered”: each buffer contains two slots. As we feed the systolic array with data from one of the slots, the other slot sends out a request to L2 for a new cache line. Once the first slot has emptied and the second slot is filled with a response from L2, the roles switch.

We store the weights for the SAE features on an on-chip SRAM memory bank. We presume that there are many tokens from different requests being processed simultaneously, as a batch. Thus, the basic mechanism of the SAE encoding is a GEMM, not a GEMV. In order to maximize the throughput of our checker, we have chosen to use a small systolic array. The systolic array is output-stationary, meaning that the partial sums of the matrix multiplication stay in place as the computation occurs. For one ‘pass’ of the systolic array, each row in the array sees a full layer of activations for one token flow sideways through it, and each column of the array sees a the full set of coefficients for one SAE feature flow downwards through

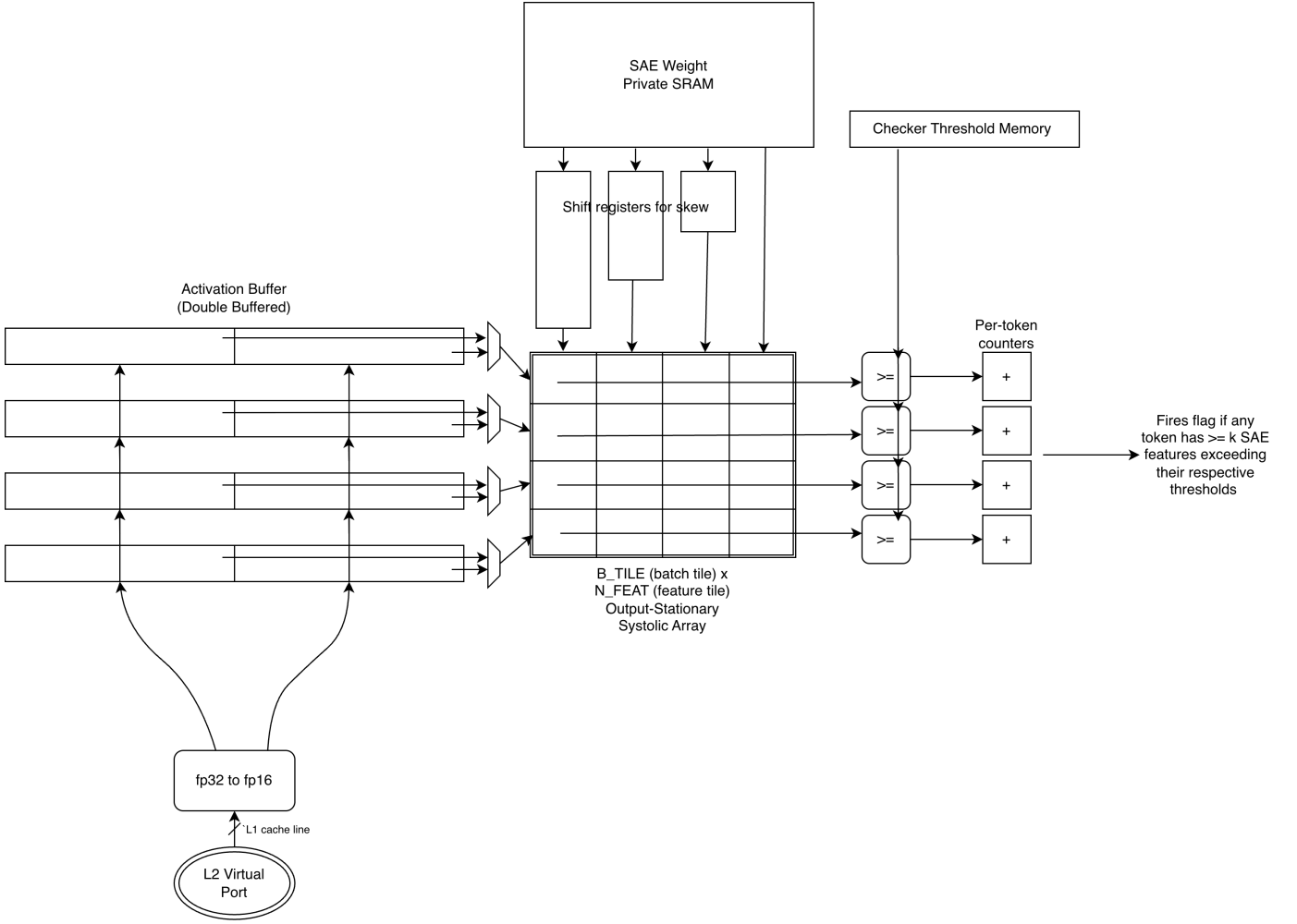


Fig. 4. Block diagram of Hardware SAE Checker Accelerator.

it. By the end of the pass, there is a full dot product stored in each cell (Processing Element). Each time two values flow into a cell, they are multiplied together and added to the stationary partial sum stored in the PE. Importantly, there is a timing skew: the values that flow through column 0 start flowing into array one cycle before the values in column 1, and the values for column 1 start flowing one cycle before column 2, and so on. The same is true for the rows. The column skew is achieved by inserting shift registers of varying length for each column, so that when a row of SAE weights are loaded from the SRAM, they reach the systolic array in a staggered fashion. The row skew is achieved by staggering the start signal for each row buffer and synchronizing the stalls between all of the rows.

After one pass of the systolic array, we have computed a full tile of SAE scores. We read these scores out of the array through row-wise shift registers. As these values flow out of the systolic array, we compare them with their respective threshold values on the fly. After all tiles of the SAE have completed, if the number of features in a row that exceed their threshold exceeds a certain constant k , then we flag

the token as misaligned.

B. Ingress Hashing Engine

The ingress hasher for the model depicted in Figure 3 places the hashing engine on the on-chip interconnect stream, snooping the write and write-address channels to digest the model weights, kernel, and metadata as they are streamed into memory. Another option would simply be to load all of the data into their respective memory storage units, make them read-only, and then read them out once again into our hashing unit. However, it is worth exploring the possibility of hashing the weights on-the-fly as they are streamed in; this could save memory bandwidth and latency. How would such a mechanism work?

We can place the hasher on the Memory-Mapped AXI stream that connects the PCIe bridge to the GPU’s VRAM. We are assuming that the host uses only one H2C (host to card) DMA channel to transmit its data, so that there is no arbitration between different channels and the AXI sees an ordered, continuous stream of data. The host would have the incentive to send the data in an ordered manner so

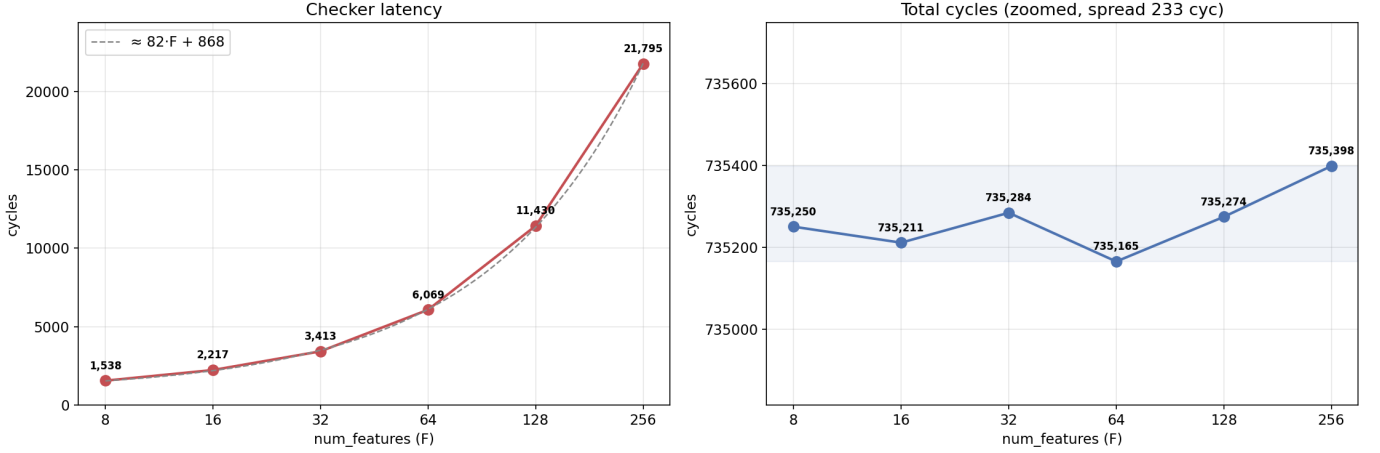


Fig. 5. Checker Latency and GEMM kernel latency (simulation cycles) plotted against Number of SAE Features. The cycle-accurate RTL simulator was run on the Vortex GPU with the addition of our checker hardware. The GPU was configured to have 4 cores, with a write-through L1 cache per core and a write-back L2 cache shared between all cores. The checker was run in parallel with a small 16x128x128 GEMM kernel. A 16x128 matrix was used as input to both the GEMM and the checker.

that it passes the hash verification scheme. The goal of the hashing unit is to be able to keep up with the throughput of the AXI stream and the PCIe interconnect that the data would come through. If we are running PCIe 4.0 x8 (8 lanes), we have a bandwidth of around 16 GB/s. The AXI stream, in order to match the bandwidth of the PCIe, would likely have something like a 512-bit bus running at 250Mhz, which would also yield 16GB/s. Our hashing unit would thus have to be capable of accepting one 512-bit block every cycle. Fittingly, SHA-256 is sequential by nature and handles one 512-bit block at a time: the algorithm starts out with a 256-bit state and compresses each 512-bit block into the state, one block at a time. Unfortunately, every 512-bit block of data must undergo 64 rounds of computation (which roughly maps to 64 cycles), and the computation for one block is entirely dependent on the 256-bit state outputted by the previous block. To handle the 64-cycle latency that each block requires, we can stripe the model into 64 streams, producing a hash for each stream. The i -th block that the hashing unit accepts will contribute to the hash for the $(i \% 64)$ -th stream. As such, by the time that we start processing the next block in a stream, the previous block in the stream will have been fully processed. This solution requires that our SHA-256 hashing unit be pipelined into at least 64 stages (which can cleanly be mapped to the 64 rounds used to process one block), and that we will have to keep track of 64 separate hashes. At the end of the transmission, we may choose to combine those 64 hashes into one final hash.

I demonstrated and tested a proof of concept for this hashing unit in SystemVerilog, using a fake AXI master and slave, and snooping the AW (write address) and W (write data) channels to read out the transmitted data. The hashing unit verifies that the write-addresses are transmitted in order, and can handle cases in which AW

and W transmissions are pipelined. Because the virtual address range of the weight memory is contiguous, but the physical addresses may not be, we place the write address snoop before the GPU address translation.

VIII. PERFORMANCE ANALYSIS

A. Checker Arithmetic

For hidden width d and F selected linear/SAE features, the checker performs approximately

$$C_{\text{check}} = dF \quad (1)$$

MACs per token, excluding bias, activation, and threshold logic. With $d = 4096$ and $F = 1024$, this is 4.19 million MACs per token. Meta’s Llama 3.1 family includes an 8B model [8]; we use a 4096-wide, roughly 10-billion-MAC/token forward pass as an architectural estimate. Under those assumptions, the checker adds about 0.042% as many MACs as the model.

The arithmetic comparison alone does not prove low latency. A dedicated checker competes for L2 bandwidth, must synchronize with token production, and may extend the critical path if output release waits for a late verdict. Kernel fusion on existing tensor cores could be more area-efficient than a specialized array. However, it is still worth exploring the merits of a dedicated hardware checker accelerator, particularly if it turns out that there is a way to verify the checker metadata without binding and verifying every kernel. Hardware specialization is most compelling when kernels are untrusted, providing kernel-independent execution and a higher level of secrecy.

We evaluate the checker’s resource contention with a GEMM kernel using Vortex’s cycle-accurate RTL simulator. The synthesizable SystemVerilog is compiled to a cycle-based C++ model using Verilator 5.028, with DRAM timing modeled by Ramulator 2. As seen in Figure 5, We

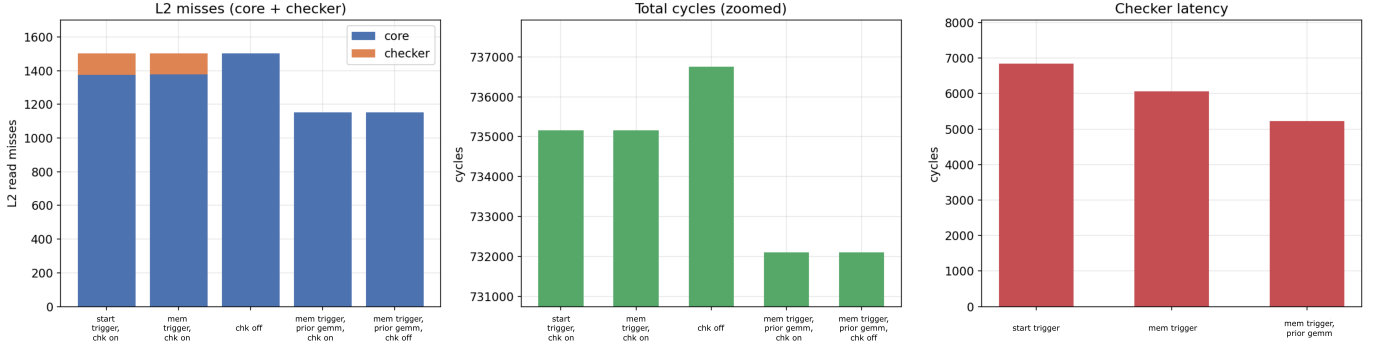


Fig. 6. Left and Middle: Miss Count and Cycle Count under various regimes. Right: Checker latency under different regimes. The cycle-accurate RTL simulator was run on the Vortex GPU with the addition of our checker hardware. The GPU was configured to have 4 cores, with a write-through L1 cache per core and a write-back L2 cache shared between all cores. The checker was run in parallel with a small 16x128x128 GEMM kernel. A 16x128 matrix was used as input to both the GEMM and the checker. 'start trigger' indicates that the checker is triggered immediately upon start of kernel execution. 'mem trigger' indicates that the checker was triggered upon the load of an address within the 128x128 GEMM matrix. 'prior gemm' indicates that a prior GEMM kernel was run whose outputs serve as the inputs to the measured GEMM.

were able to demonstrate that as checker computation and latency grows, the latency of the kernel (we used a small GEMM with an output dimension of 128 and an input dimension of 128) stays roughly constant, indicating minimal L2 bandwidth or memory bandwidth contention.

Figure 6 demonstrates that activating the checker might actually decrease the latency of the kernel in which it is run on. From the left bar chart, we observe that with the checker activated, the checker absorbs some of the misses that the core would have taken (namely those in the input matrix). As such, it acts as a sort of prefetcher for the kernel. In the middle diagram, we observe that turning the checker off increases the latency of the GEMM kernel. When we have a prior kernel that loads all the values for the input matrix into the L2 cache, we observe that the checker has little effect on the cycle count of the GEMM, indicating minimal L2 contention.

IX. DISCUSSION AND LIMITATIONS

Alignment isn't just a hash. The architecture can guarantee that a particular checker ran on a particular admitted execution, but it cannot guarantee that the checker accurately captures human values or is free from false positives.

Confidentiality Conflict. The enforcement of alignment checkers on models requires that model weights are made known at least to the trusted third party so that the checker weights can be generated. The inspection of kernels and model weights may conflict with intellectual property rights over closed-source model weights and runtimes.

Dynamic models remain difficult. Behaviors like dynamic weight sharding, continuous fine-tuning, and dynamic quantization expand the manifest state space.

X. CONCLUSION

Activation-based monitors may detect safety-relevant internal behavior that output filters miss, but they are

ineffective if an autonomous workload or malicious operator can disable them. A GPU root of trust offers direct visibility into model state and a smaller mutable TCB. Our proposed architecture signs and measures model weights, checker parameters, kernels, and metadata; locks verified memory; confines executable pages; and gates output on a checker verdict.

The analysis reveals an asymmetry: computing a useful probe can be cheap, while proving that the probe observed the real computation is expensive and restrictive. Whole-kernel measurement is a defensible starting point, but realistic deployment depends on more flexible forms of certified execution, rack-scale enforcement strategies, checker robustness, and careful governance.

REFERENCES

- [1] N. Goldowsky-Dill, B. Chughtai, S. Heimersheim, and M. Hobbhahn, "Detecting strategic deception using linear probes," *arXiv preprint arXiv:2502.03407*, 2025. DOI: [10.48550/arXiv.2502.03407](https://doi.org/10.48550/arXiv.2502.03407).
- [2] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse autoencoders find highly interpretable features in language models," *arXiv preprint arXiv:2309.08600*, 2023. DOI: [10.48550/arXiv.2309.08600](https://doi.org/10.48550/arXiv.2309.08600).
- [3] N. Panickssery, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner, "Steering llama 2 via contrastive activation addition," *arXiv preprint arXiv:2312.06681*, 2023. DOI: [10.48550/arXiv.2312.06681](https://doi.org/10.48550/arXiv.2312.06681).
- [4] P. Han, C. Qian, X. Chen, Y. Zhang, H. Ji, and D. Zhang, "SafeSwitch: Steering unsafe LLM behavior via internal activation signals," *arXiv preprint arXiv:2502.01042*, 2025. DOI: [10.48550/arXiv.2502.01042](https://doi.org/10.48550/arXiv.2502.01042).

- [5] J. Mickens, S. Radway, and R. Netravali, “Guillotine: Hypervisors for isolating malicious AIs,” in *Proceedings of the 2025 Workshop in Hot Topics in Operating Systems*, ACM, 2025, pp. 18–26. DOI: [10.1145/3713082.3730391](https://doi.org/10.1145/3713082.3730391).
- [6] M. Russinovich, Y. Cai, and A. Salem, “Hey, that’s my model! introducing chain & hash, an LLM fingerprinting technique,” in *International Conference on Learning Representations*, Originally posted as arXiv:2407.10887 in 2024, 2026. DOI: [10.48550/arXiv.2407.10887](https://doi.org/10.48550/arXiv.2407.10887).
- [7] NVIDIA Corporation, *NVIDIA Attestation*, Documentation accessed July 2026, 2026. [Online]. Available: <https://docs.nvidia.com/attestation/index.html>.
- [8] Meta AI, *Llama 3.1 model card*, Accessed July 2026, 2024. [Online]. Available: https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md.